

1. ERD Design & Normalisation

- Can you review the ER Diagram, particularly the blue section focused on Myki card touch-on behaviour?
 - Is the relationship between Transaction and Trip correctly set as optional (0..1), considering some transactions may be top-ups or pass purchases?
 - The Scanner sits between Transaction and DeviceLocation — does this placement make logical and structural sense?
 - Is it still considered 3NF if I add redundancy like storing stop_station_id in DeviceLocation for performance?
 - Please confirm if all tables follow 3NF, except DeviceLocation which intentionally includes controlled denormalisation.
 - If VehicleStopLog updates every 30 seconds and synchronises current_stop_station_id in DeviceLocation, is this approach too computationally expensive?
 - Would it be better to store vehicle_id or stop_station_id directly in the Scanner table for efficiency?
 - When a scanner detects a Myki card, how is the card number identified — is it automatic in real systems?
-

2. Indexing & Partitioning

- Can you suggest which tables should be indexed to optimise the touch-on process?
 - Should Scanner.device_location_id be indexed if it's already a foreign key pointing to a primary key?
 - Should indexing in MykiPass use card_id or pass_id, considering that pass_id is the PK and referenced by MykiCard?
 - Which columns should be used for horizontal partitioning based on datetime fields?
 - What are the operational differences between monthly and daily partitioning in SQL Server?
 - Would vertical partitioning benefit tables like CustomerAccount, and how should I structure it?
-

3. Stored Procedures & Business Logic

- Can you help me describe the internal flow of usp_TouchOn, including when it calls usp_CheckTapCardStatus, usp_InsertTripOnTouchOn, and usp_ProcessTouchOffTransaction?
 - What logic is used to distinguish between touch-on and touch-off in the Trip table?
 - How should the stored procedure return success or error messages to simulate scanner feedback?
-

4. Report Section Writing

- Can you help reword the “Purpose” and “Scope” sections of the specification to sound more realistic and industry-appropriate?
 - How should I describe the logical and physical schema in Section 2.5?
 - Should I mention the use of dbo. schema prefixes and naming conventions like usp_, udf_, and vw_?
 - How should I phrase the decision to place the ERD in the appendix instead of in the main body?
 - How can I properly justify the denormalisation in DeviceLocation under physical design?
 - What’s the difference between 2.5.2 Physical Design and 2.5.3 Physical Structure, and how should I split the content?
 - Can you help me write each subsection in Chapter 3 (e.g., Recovery, DBMS Info, Error Handling)?
 - How do I reference long storage requirement calculations in the appendix from Section 3.3?
 - What should I write in Section 3.6 Database Interfaces if no external integration is implemented?
-

5. Data Dictionary & Documentation Tools

- Can you generate a data dictionary in Excel from my DDL file?

- Can you extract and list column details like name, datatype, PK, FK, nullable, and descriptions?
 - Should constraint details like CHECK (balance >= -10) be explained in the main spec or elsewhere?
 - Can you help format 2.6 Data Dictionary to reference Appendix A?
-

6. Testing, Execution & Special Instructions

- How should I document the order and purpose of each .sql script (e.g., partition setup, schema creation, index creation)?
 - What should I include in the “Testing Procedures” section to simulate scanner interactions and card taps?
 - Can I mention the GitHub repo for version control under Recovery documentation?
 - Where and how should I describe the trigger on vwCustomerAccount that handles inserts across partitioned tables?
-

7. Partition Design & Dependencies

- How should I write about the partition function and scheme used (pf_Monthly, ps_Monthly) in the description section?
 - Can you explain partition interdependencies across tables like Trip, CardTransaction, and VehicleRealTimeLog?
 - How do I describe the use of a single filegroup for all partitions in the educational SQL Server setup?
-

8. Storage Estimation & Capacity Planning

- Estimate per-record storage requirements based on field-level SQL Server data type sizes.

- Calculate row overhead and index impact for Trip, CardTransaction, and high-frequency logging tables.
 - Estimate annual growth of partitioned tables using simulated trip volume data.
 - Include static/slowly-growing tables like MykiCard, MykiPass, CustomerAuth, and CustomerProfile.
 - Consolidate per-table estimates into a unified 2023 baseline.
 - Forecast 2033 storage using Victorian population growth, CAGR, and per-capita trip assumptions.
 - Describe forecasting methodology clearly and quantify total projected growth.
 - Present final results in a single concluding paragraph summarising expected annual storage impact.
-

9. T-SQL & Schema Implementation

- Designed and implemented all required SQL DDL for the core schema, divided into two major domains:
 - **Blue section (Myki Card system logic):** included CustomerAccount (via CustomerAuth and CustomerProfile through vertical partitioning), MykiCard, MykiPass, Trip, CardTransaction, CardType, and FareType.
 - **Green section (Vehicle operations and tracking logic):** included StopStation, Route, RouteStop, VehicleType, Vehicle, VehicleRun, VehicleRealTimeLog, and VehicleStopLog.
- Applied specific field-level enhancements and structural decisions:
 - Changed Trip.touch_on_time and touch_off_time to use DATETIME2(0) for improved timestamp precision.
 - Renamed Transaction table to CardTransaction to avoid reserved word conflicts and better reflect semantic purpose.
 - Required CardTransaction.amount to be positive via a CHECK constraint.
 - Enforced FK relationships in MykiCard to CardType, and in Trip to FareType.
- Defined vertical partitioning on the CustomerAccount entity:
 - Split into CustomerAuth (frequently accessed auth data) and CustomerProfile (infrequently updated personal data).

- Created vwCustomerAccount to provide unified access, and trg_vwCustomer_IOI trigger to support insert operations across both base tables.
- Created lookup tables for standardising value references:
 - CardType: e.g., full_fare, concession, child, senior.
 - FareType: e.g., default_fare, free_2hours, free_mykipass, concession_weekend.
- Implemented custom constraints:
 - Enforced an XOR constraint on DeviceLocation, ensuring exactly one of vehicle_id or stop_station_id is non-null.
 - Disabled cascading deletes between DeviceLocation and Scanner to prevent unintentional data loss.
 - Defined composite primary keys on entities like RouteStop (route_id, stop_station_id, direction) for sequence-aware stop management.
- Eliminated redundant or unnecessary tables (e.g., Bus) in favour of generalised VehicleType, improving schema modularity and reducing duplication.
- Introduced tracking architecture separation:
 - VehicleRealTimeLog: logs raw GPS coordinates at high frequency (e.g., every 30 seconds).
 - VehicleStopLog: derived stop arrival information inferred from GPS data and stop geolocation, simulating post-processing logic.
- Explored real-time vs. derived architecture questions:
 - Evaluated the pros and cons of computing nearest stop live vs. caching current_stop_station_id in DeviceLocation.
 - Discussed implementation options using triggers, streaming logic, or external batch jobs (though not implemented).
- Translated final DDL (including comments) into clean, formal English, following consistent naming conventions and documentation standards.
- Developed modular T-SQL logic for fare type determination:
 - Implemented as a multi-step procedure or function that considers:
 - Time since last trip (2-hour rule)
 - Active MykiPass presence (pass_id in MykiCard)
 - Card type (CardType logic, including free fares for seniors on holidays)
 - Zone movement based on StopStation.zone values
 - Daily caps, weekday/weekend fare policies from Calendar

- Structured fallback rules for fare classification (default_fare, default_weekend, etc.) when none of the specific rules apply.